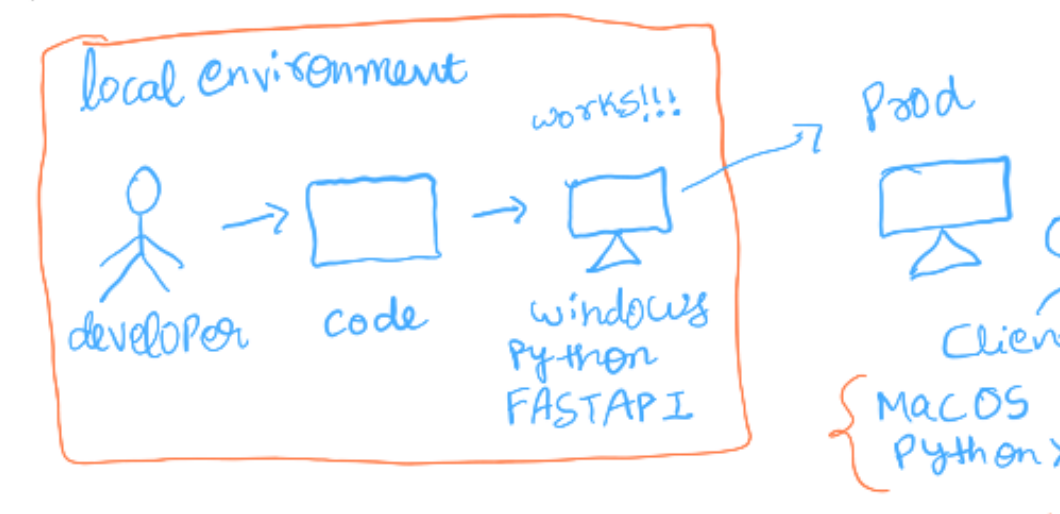
Module-7 : Docker Fundamentals

21/02/2026 * Python for DevOps → Revisit

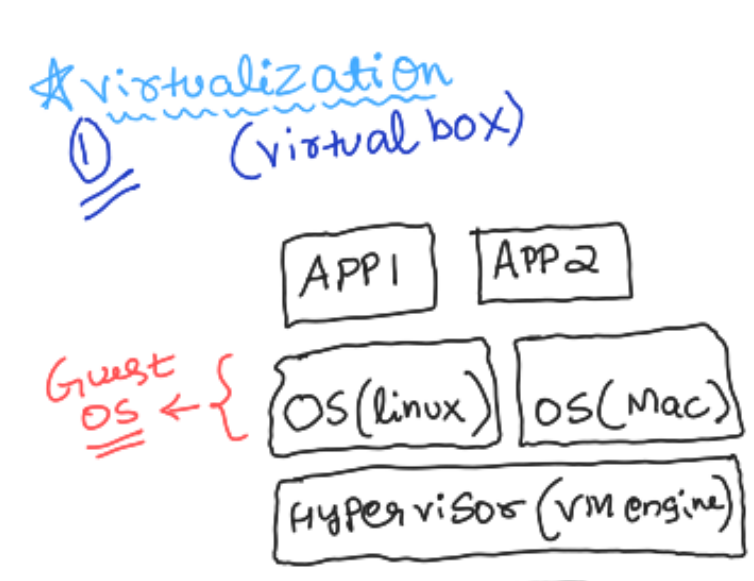


Problem ↓
→ OS mismatch
→ libraries & dependencies mismatch
→ Commands & Instrns unclear

Solution ↓
→ local environment → Replicate → virtual environment

* virtualization

① (virtual box)



isoimage X

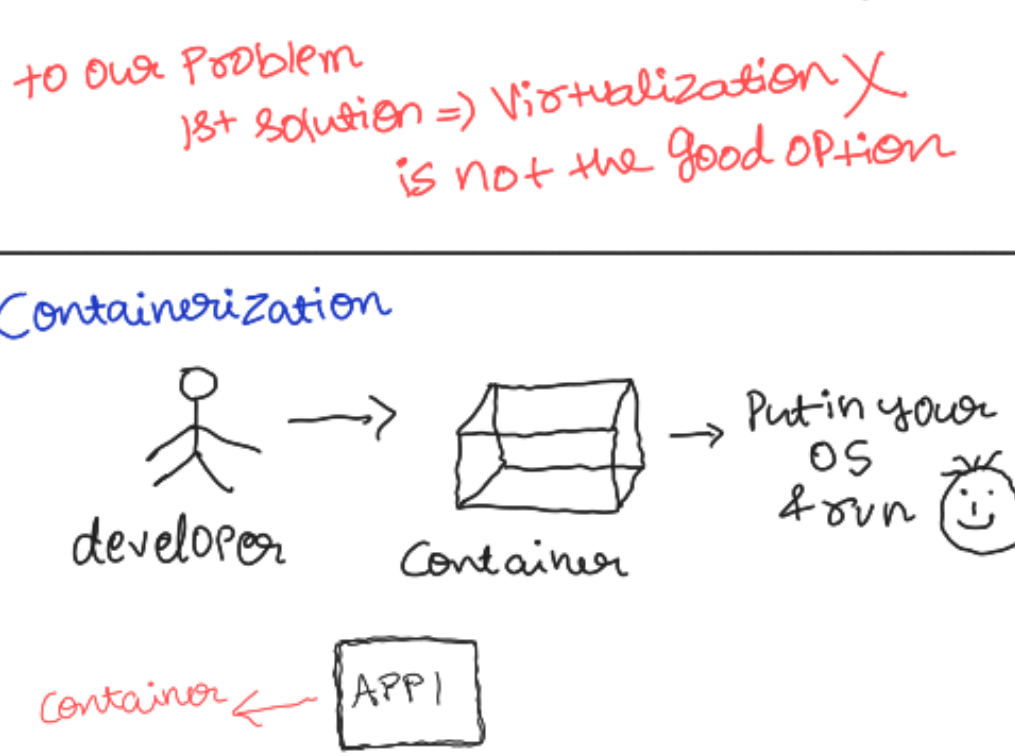
Virtualization is a → Process of adding a hypervisor on top of the Host OS so that we can install/use new OS (guest OS) which can take shared resources/ dedicated resources from your own device which can be used to run any appln.

(windows, Linux, macOS)

Issues ↓
- Allocated RAM, Storage
- System can get slow
- duplication of OS
- Performance issues
- Overutilization of resources
- High cost
- Storage issues
- Crashing repeatedly

So to our Problem
1st solution ⇒ Virtualization X
is not the good option.

② Containerization



- no Performance issues
- no duplication of OS

So, now when APP1 is in container
↳ and then that APP1 will ask the Docker engine that it needs diff OS (linux) etc
↳ And Docker engine will auto convert do the process for APP1
↳ And once the task of APP1 is done then the used storage/resources → again come to normal (bcoz its shared)

* Docker ↓

Docker → enterprise X
Docker → community ✓ (free)

Docker ⇒ containerization tool

- Docker client → CLI tool used to send commands to Docker daemon
- Docker engine → core runtime that builds, runs & manages containers
- Docker Daemon → background service (dockerd) that handles API req, manages images, containers, network, volumes.
- Docker Registry → service for storing & distributing images

{Docker client → Docker daemon → Docker registry}

Docker Engine (daemon + Rest API + CLI)

Docker client → CLI = docker CLI
Docker client → GUI = docker desktop
} Same docker daemon

Docker ⇒ uses a central daemon (rootful)
↳ uses containerd under the hood
→ uses a root owned daemon but supports rootless mode

Podman ⇒ doesn't have central daemon
↳ runs containers directly
↳ more secure & safer
↳ rootless (running containers without root/ admin privileges)

containerd ⇒ engine room
↳ don't usually use it directly
↳ It pulls images, manages containers, used by Docker & Kubernetes
↳ rootful

* Download
↳ Docker Desktop
includes Docker CLI (client), Docker engine (daemon), Docker compose

Commands to verify
→ `docker --version` (you get the version of the docker client)
→ `docker version` gives both client & server version

Client → docker CLI version
Server → docker engine (daemon) version

Client - docker cli
Server → engine version, containerd version, runc version

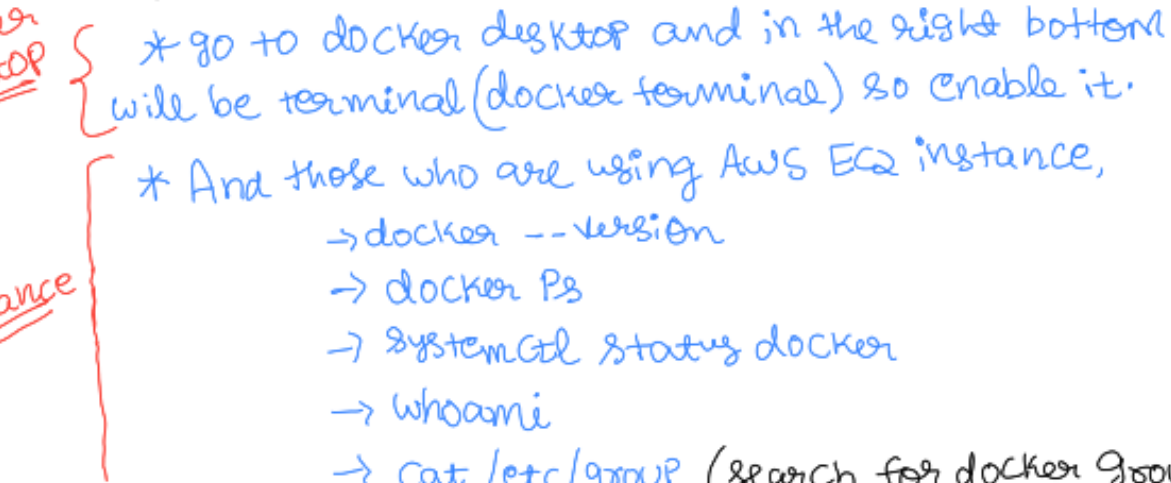
* Create account in DockerHub

Practical ↓ EC2
→ launch AWS instance
→ then ssh into newly created EC2 instance
→ Update the instance (`sudo apt-get update`)
→ Next install docker in instance (`sudo apt-get install docker.io`)
→ check version (`docker --version`) ↳ Install the CLI version on Ubuntu/ install docker engine on CLI

windows/MacOS ⇒ docker desktop
Linux (developers) ⇒ docker desktop (optional)
Linux (servers) ⇒ only docker engine + docker CLI

Formula ↓
{dockerfile → image → container}

* container
↳ ultimate final goal we need
↳ they are made from docker image



dockerhub → Public repo where docker images are stored, shared and downloaded
docker file → text file containing instrns to build a Docker image
docker image → read only blueprint/template used to create Docker containers
docker container → running instance of a docker image that executes an appln.

* 1st task → Create a Container of Ubuntu

docker desktop { * go to docker desktop and in the right bottom there will be terminal (docker terminal) so enable it.

AWS ec2 instance { * And those who are using AWS EC2 instance,
→ `docker --version`
→ `docker ps`
→ `systemctl status docker`
→ `whoami`
→ `cat /etc/group` (search for docker group)
add user to the docker grp { → `sudo usermod -aG docker ubuntu`
→ `cat /etc/group` (now see in the docker group whether ubuntu user got added/not)
→ `newgrp docker`
→ `docker ps`

Now, the 1st task ↓
① * In docker desktop ⇒ {`docker run -itd ubuntu`}
docker → docker CLI command
run → create and start a new container
-i → interactive (so we can interact with the container)
-t → tty (allocates a terminal inside the container)
-d → detached (runs the container in the background)
ubuntu → docker image used to create the container (downloaded from dockerhub if not available locally)

* Now check using ⇒ [`docker ps`]
It will generate a long id ⇒ (Container Id)

* Next in docker desktop check the [Containers tab] you will see the container created

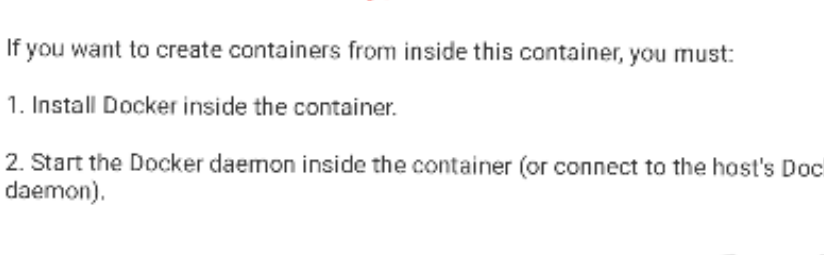
{* Same you can do in AWS EC2 instance running all cmds.}

* Next to enter inside the running container
[`docker exec -it <cont id> bash`]
docker → docker CLI command
exec → execute a command inside an already running container
-i → interactive mode
-t → allocate a terminal
cont id → ID/name of the running container
bash → start the bash shell inside the container

* Next, now we are inside Ubuntu so we can do
→ `apt-get install nginx`
→ `apt-get update`
→ `apt-get install nginx`
→ `systemctl status nginx`
→ `service nginx status`
→ `service nginx stop`
→ `service nginx status`

* Next, in docker desktop we can pause, stop, delete the container when not needed.

* We can also have docker inside docker (DinD)



If you want to create containers from inside this container, you must:
1. Install Docker inside the container.
2. Start the Docker daemon inside the container (or connect to the host's Docker daemon).

* To exit from running container ⇒ [`exit`]

* 2nd task → container of nginx
② → `systemctl status nginx` (in aws ec2 instance)
→ If not present, then do install but create container of nginx
→ use cmd {`docker run -d -p 80:80 nginx`}
docker → docker CLI command
run → create & start a new container
-d → detached mode (background)
-p 80:80 → Port mapping
80:80 host port / container port
nginx → docker image used to create the container
→ Next, check using {`docker ps`}
→ Next, go to EC2 instance & copy the Public IP and paste in the browser (you should be able to see welcome to nginx!)

* boom!! without doing [`sudo apt-get install nginx`] we are running nginx in the container

* we can do in docker desktop also
{`docker run -d -p 80:80 nginx`}
→ now you can go to (localhost:80) you will see nginx page

* Port Mapping ↓

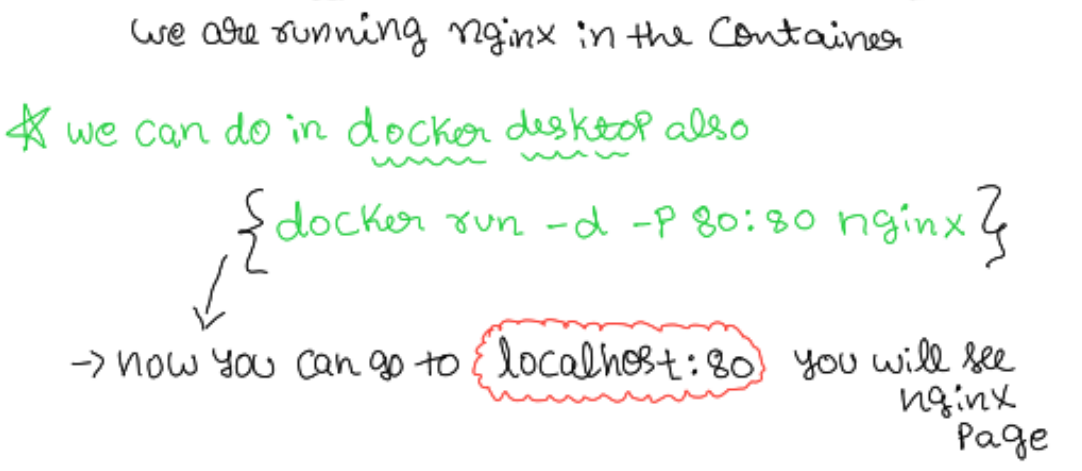


Image
Host OS
80:80
Container exposes its ports so Host OS can access it
Container
* virtual env
→ code (we get from image)
→ dependencies (OS, nginx)
→ Commands
we get from image
this container takes from Host OS

* Next, create Container of Hello World
③ {`docker run hello-world`}

docker run = Create + Start
docker exec = Enter (or run a cmd in) a running container

* Some useful Commands
→ `docker ps` (only shows running containers)
→ `docker logs <cont id>`
→ `docker stop <cont id>`
→ `docker ps -a` (shows running & stopped containers)
→ `docker start <cont id>`
→ `docker rm <cont id>` (To remove completely (1st you need to stop the containers & then you can remove))

* Container of mysql ↓
{`docker run -d --name mysql-demo -e MYSQL_ROOT_PASSWORD=test@123 mysql`}
docker → docker CLI
run → create & start a new container
-d → run the container in the background
--name mysql-demo → give the container the name
-e → set an environment variable inside the container
Pwd → sets the MySQL root user's Pwd
mysql → official mysql docker image

→ next run, [`docker ps`]
→ next run, [`docker exec -it <cont id> bash`]
→ next run, [`mysql -u root -p`]
→ Enter the Pwd
→ to exit, run [`exit`]
→ [`docker ps`]
→ [`docker stop <cont id>`]
→ [`docker rm <cont id>`]

* Container of MongoDB ↓
→ [`docker run -d mongo`]
→ [`docker ps`]
→ [`docker exec -it <cont id> bash`]
→ [`show dbs;`]
→ [`docker ps`]
→ [`docker stop <cont id> && docker rm <cont id>`]

* docker Project (Basically creating Dockerfile) so that we can create image from that docker file
→ `mkdir docker-projects`
→ `cd docker-projects`
→ have [index.html, install_nginx.sh] → custom

So, create a file inside docker-projects called Dockerfile

So, we shall write a Dockerfile ↓

```
FROM nginx
WORKDIR /app
COPY index.html /var/www/html
EXPOSE 80
```

Now, we have Dockerfile ready now we can build the image from Dockerfile ↓
{`docker build -t devops-nginx-demo .`}

docker run ⇒ we get Container Id
docker build ⇒ we get Image Id

Now, we have Docker Image ready now we can have/build the container from image ↓
{`docker run -d -p 80:80 devops-nginx-demo:latest`}

→ next, {`docker ps`}
→ Now, copy the instance Public IP and paste in the browser
→ Now, go into nginx container by
{`docker exec -it <cont id> bash`}
→ do {`ls`}
→ `cd /`
→ `cd var/www/html`
→ `grep -i -r "Welcome to nginx!" /`

The path you get is, where index.html is present
{/usr/share/nginx/html/index.html}

→ exit

Now, go to Dockerfile and make changes of path ↓

```
FROM nginx
WORKDIR /app
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

① → `docker build -t devops-nginx-demo .`
④ → `docker run -d -p 80:80 devops-nginx-demo:latest`

→ Now, go to ec2 instance & paste its Public IP in the browser

before this
② * `docker stop <cont id>`
③ * `docker rm <cont id>`

* To see docker images we {`docker images`}

→ for verify, do {`docker exec -it <cont id> bash`}

* Creating Dockerfile for devops-utilities-api (Python for devops)

→ `cd Python-for-devops/Projects/devops-utilities-api/`
→ `vim Dockerfile`

```
FROM Python:3.14
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8000
CMD
```

→ Next, {`docker build -t devops-utilities-api .`}
→ Next, {`docker run -d -p 8000:8000 devops-utilities-api`}
→ Next, {`docker ps`}
→ Next, go to ec2 instance → security groups → edit inbound rules → Custom TCP → 8000 → My IP → Save
→ Next, copy the ec2 instance Public IP and then paste in the browser
<Public IP>:8000
<Public IP>:8000/metrics

* Creating Dockerfile for Java Project ↓

* Docker Quiz ↓

1] VM need full guest OS
Containers share host OS Kernel

2] To remove
docker container → docker rm command
docker image → docker rmi command

3] In Dockerfile
ADD → downloads files from a URL
COPY → only copies from local filesystem

4] EXPOSE ⇒ serves as documentation to indicate which Ports the app listens to

5] docker Performs ⇒ OS-level virtualization

6] To search images in the Docker hub ⇒ docker search <image name>

7] Owner/author of the image ⇒ MAINTAINER

8] ENTRYPOINT runs first
CMD is appended as arguments to it

* Dockerfile instructions

FROM → specifies the base image

LABEL → adds metadata to the image

WORKDIR → sets the working directory

COPY → copies files from the local machine into the image

ADD → similar to COPY, but can also extract local .tar files and dwnld from URLs.

RUN → executes commands while building the image

ENV → sets environment variables

EXPOSE → documents which Port the application listens on.

CMD → specifies the default command when the container starts

ENTRYPOINT → specifies the main executable of the container

USER → runs the container as a specific user

VOLUME → creates a mount Point for persistent data